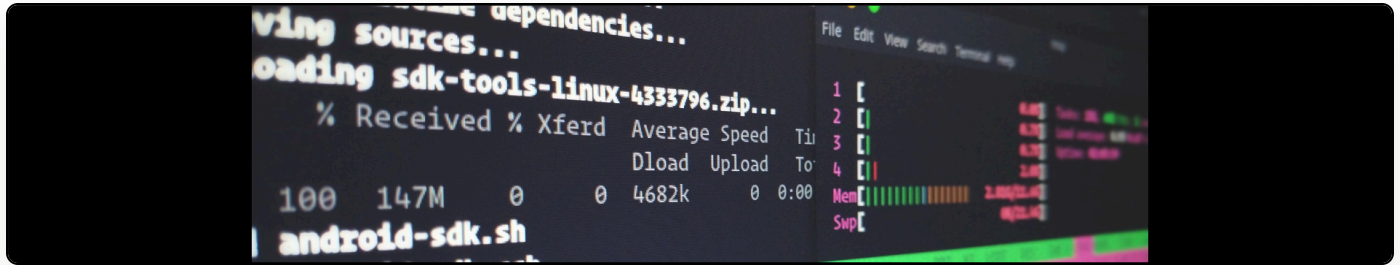


Selecting specified text ranges in a browser using javascript and xpath

Cite as: Martin Paul Eve, "Selecting specified text ranges in a browser using javascript and xpath", *eve.gd: Martin Paul Eve*, June 26, 2016, <https://doi.org/10.59348/w33g5-qyt55>.



Continuing [my post from yesterday](#), one of the interface components that we want to work is that, when a user clicks a paragraph, the first sentence is selected so that they can immediately begin translating, seamlessly hitting enter to move to the next sentence etc. I've been working on this while my colleague has been creating the sidebar interface.

It turns out, though, that selecting a specified sentence of text in javascript is not easy. Very few browsers support selecting TextNodes in their raw range functionality.

Fortunately, I *know* that hypothes.is can do this, because it is core to what they have achieved. The challenge was understanding how we could repurpose this for our own effort.

The initial CoffeeScript that I wrote to handle this is as follows:

```

Annotator = require('annotator')
$ = Annotator.$
xpathRange = Annotator.Range
Util = Annotator.Util

makeSentenceSelection: (event = {}) =>
  # Get the currently selected ranges.

  desiredText = event.target.innerText || event.target.textContent
  desiredText = desiredText.split('.')[0]

  full_xpath = Util.xpathFromNode($(event.target), document)

  data = {
    start: full_xpath
    startOffset: 0
    end: full_xpath
    endOffset: desiredText.length
  }

  anchor = new xpathRange.SerializedRange(data).normalize(document)

  window.getSelection().removeAllRanges()
  window.getSelection().addRange(anchor.toRange())

```

To talk through how this works... Firstly we establish a function called `makeSentenceSelection` that is an event handler. Elsewhere, I've bound the document's click event to this function. We then strip out the text content that we want to match (by doing a string split on the "." character and returning the zero-element in the array) and store it in "desiredText". After this, I use a util class from the Annotator shipped with `hypothes.is` to create an xpath expression that uniquely identifies the clicked node.

Once this is done, I bundle up a data dictionary that will tell the `SerializedRange` to start at the xpath element, end at the xpath element, start counting text from position 0, and highlight the length of the earlier-calculated `desiredText`. I then create a new `SerializedRange` object, passing it this dictionary, and tell it to create a "normalized" (that is, browser-compatible version) of itself. Finally, I remove all other selected ranges and then add the value of `anchor.toRange()` as the current selection.

And... tada! This succesfully highlights, in most cases, the first sentence of a clicked paragraph.

Now, there's much more still to do here, but this is a good first step on our interface front.